

RAPPORT PROJET IA

- **Table des matières**

1. Implémentation des joueurs de base
 - i. Choix aléatoire sans validation
 - ii. Choix aléatoire avec validation
 - iii. Premier coup valide
2. Minimax
 - i. Stratégie Minimax
 - ii. Méthode minimax
 - iii. Tableau évalué et simulé
 - iv. Evaluation du tableau
 - v. Mesures de performance
3. Alpha-Beta
 - i. Principe de l'élagage Alpha-Beta
 - ii. selectMove
 - iii. alphaBeta
4. MCTS
 - i. Recherche Monte-Carlo
 - ii. Structure d'un nœud
 - iii. Constantes et UCB1
 - iv. selectMove
 - v. Sélection
 - vi. Expansion
 - vii. Simulation
 - viii. Backtracking
5. Tests
6. Tests additionnels
7. Performance et comparaisons
 - i. AlphaBeta et Minimax contre un humain
 - ii. AlphaBeta vs Minimax
 - iii. MCTS contre un humain
 - iv. MCTS vs AlphaBeta, constante d'exploration
8. Optimisations faites
 - i. AlphaBeta
 - ii. Minimax
9. Tests après optimisations
 - i. Minimax
 - ii. AlphaBeta

- **Implémentation des joueurs de base**

1. Choix aléatoire sans validation

C'est une stratégie simple où on choisit aléatoirement la ligne, la colonne et la valeur (RandomMoveNoValidationStrategy, l.38, l.39, l.65). Tant qu'on ne trouve pas une case libre, on continue à générer des coups aléatoires. Pour ne pas boucler infiniment, on a introduit une valeur de plafond pour la boucle (on a décidé à l'initialiser à la taille du plateau de jeu au cube). Au cas où un nœud n'est pas trouvé, on a introduit une recherche systématique.

2. Choix aléatoire avec validation

La stratégie a le même fonctionnement que le choix aléatoire sans validation, à part que on fait appel à la méthode isValidMove de la classe Player pour vérifier la validité du coup généré. Le plafond pour la boucle est aussi introduit et est égal à la taille du plateau au cube. Une collection est faite pour mélanger les cellules libres.

3. Premier coup valide

On commence par la première ligne et première colonne. On parcourt ensuite la totalité de la première ligne et on passe à la deuxième. On fait ainsi jusqu'à ce qu'on trouve un coup valide. Si aucun coup n'est trouvé, on renvoie null.

- **Minimax**

1. Stratégie Minimax

Cette stratégie implémente l'algorithme minimax. On commence par initialiser la profondeur jusqu'où on cherche à 3. La méthode selectMove implémente l'algorithme lui-même. On met la variable comportant la meilleure valeur à moins infini. On crée aussi un tableau évalué et simulé (EvaluatedSimulatedBoard) à partir du tableau réel au moment où on appelle la méthode. Pour tous les coups valides existants, on applique ce coup, on appelle la méthode minimax, mais cette fois-ci avec le joueur opposé (pour lequel on cherche à minimiser le score) et une profondeur plus petite, pour recevoir la bestValue, on et on retourne le tableau dans l'état d'avant. On cherche ainsi la meilleure valeur retournée et on retourne le coup correspondant.

2. Méthode minimax

La méthode minimax (l.209) implémente l'algorithme récursif avec les paramètres suivants :

- **board** : Le plateau simulé courant
- **depth** : La profondeur restante d'exploration
- **isMaximizingPlayer** : Booléen indiquant si le joueur courant maximise son score
- **player** : Référence au joueur IA

Le fonctionnement suit ces étapes :

- a. Incrémentation du compteur de nœuds visités
- b. Condition d'arrêt (profondeur nulle ou plateau complet)
- c. Génération des coups valides

- d. En accord avec la variable isMaximizingPlayer, on initialise la bestValue et moins infini ou plus infini
- e. Évaluation réursive de chaque coup possible, en diminuant la profondeur et en inversant la variable du joueur maximisant
- f. Annulation du coup après évaluation (backtracking). La valeur précédente de la cellule est gardée dans prevValue
- g. Sélection du meilleur score (max pour le joueur, min pour l'adversaire)

3. Tableau évalué et simulé

La classe EvaluatedSimulatedBoard étend CSudokuBoard avec :

Fonctionnalités ajoutées :

- Suivi des cases vides par ligne/colonne (tableaux zerosInRows et zerosInColumns)
- Évaluation heuristique (eval)
- Historique du dernier coup joué (lastMove)

Mécanisme de copie :

Constructeur de copie (l.43) qui duplique :

- L'état du plateau
- Les contraintes
- Les compteurs de cases vides

4. Evaluation du tableau

La méthode setValue (l.178) met à jour l'évaluation selon :

Points de base :

- Valeur numérique du coup joué (1-9)

Bonus :

- Taille_plateau² pour :
 - Compléter une ligne (rowFilled)
 - Compléter une colonne (colFilled)
 - Compléter un sous-carré (subgridFilled)

Mise à jour différentielle :

- Score augmenté pour le joueur actif
- Score décrémenté pour l'adversaire (via eval -= delta)

Gestion des états :

- Mise à jour des compteurs de cases vides
- Vérification de la validité des coups
- Journalisation des erreurs potentielles

5. Mesures de performance

- Profondeur maximale fixée à 3 pour équilibrer la performance et la précision
- Comptage des nœuds visités
- Chronométrage des recherches

- **Alpha-Beta**

1. Principe de l'élagage Alpha-Beta

La stratégie Alpha-Bêta améliore l'algorithme Minimax en éliminant les branches inutiles de l'arbre de recherche. Les caractéristiques principales sont :

- Profondeur fixée à 3
- Les variables α et β , correspondants respectivement aux meilleures valeurs garanties pour le joueur MAX et MIN
- Observateur AlphaBetaPruningObserver pour tracer le nombre de nœuds visites, coupes α et coupes β

2. selectMove

Au début on suit le même principe que pour MiniMax, puis on initialise α à moins infini et β à plus infini. Pour chaque coup valide, on applique récursivement l'algorithme alpha-beta, en appliquant le backtrack ensuite. La bestValue est modifiée et alpha prend la valeur max entre alpha courant et la valeur actuelle après modification.

3. alphaBeta

Pour le joueur MAX (resp. MIN) :

- Pour chaque coup valide, on évalue le terrain
- On applique le backtrack
- On prend la valeur maximale (resp. minimale) trouvée pour bestvalue
- On applique la coupe beta (resp. alpha) si $\beta \leq \alpha$

- **MCTS**

1. Recherche Monte-Carlo

La Recherche Arborescente Monte-Carlo (MCTS) est un algorithme de recherche adapté aux jeux possédant un grand facteur de branchement, comme le CSudoku. Au lieu d'explorer exhaustivement l'arbre de jeu comme Minimax ou Alpha-Beta, MCTS utilise l'échantillonnage aléatoire (simulations) pour estimer la valeur des coups, construisant itérativement un arbre de recherche concentré sur les zones les plus prometteuses.

2. Structure d'un nœud

- state: Un EvaluatedSimulatedBoard représentant l'état du jeu à ce nœud
- parent : Une référence vers le nœud parent (null pour la racine)
- children : Une liste d'objets MCTSStrategy représentant les nœuds enfants
- move : L'objet Move qui a mené de l'état parent à l'état de ce nœud
- visitCount: Le nombre de fois que ce nœud a été inclus dans un chemin durant le processus MCTS
- totalScore : Le score cumulé obtenu des simulations qui sont passées par ce nœud

- `unTriedMoves` : Une liste des coups valides depuis l'état de ce nœud qui n'ont pas encore été faits en nœuds enfants

3. Constantes et UCB1

- La constante d'exploration est introduite pour la formule UCB1, dans notre cas $\sqrt{2}$ (l.25)
- La limite de simulation est fixée à 100000

4. `selectMove`

La methode suite 4 etapes pour le nombre de simulations fixe:

- selection
- expansion si le nœud choisi n'est pas une feuille
- simulation
- propagation en arriere
- le meilleur nœud est choisit (l.265). Cela se fait en examinant les enfants du nœud racine. Le coup menant au nœud enfant avec le `visitCount` le plus élevé est sélectionné.

5. Sélection

En partant du nœud racine, l'algorithme parcourt l'arbre vers le bas en sélectionnant répétitivement le "meilleur" enfant selon la formule UCB1

6. Expansion

Si le nœud sélectionné n'est pas terminal et n'est pas complètement étendu, un coup est choisi aléatoirement parmi ses `unTriedMoves`. Ce nouveau nœud enfant est ajouté à la liste `children` du nœud sélectionné, et le coup choisi est retiré de `unTriedMoves`

7. Simulation

Depuis le nœud choisi lors de la phase d'expansion, une partie simulée est jouée jusqu'à atteindre un état terminal. Les coups durant la simulation sont choisis aléatoirement. Une fois la simulation terminée, un score final est calculé basé sur l'état terminal. Le score final est retourné en prenant la difference du score des joueurs.

8. Backtracking

Le résultat (score) de la simulation est propagé en remontant l'arbre depuis le nœud où la simulation a commencé, jusqu'au nœud racine.

• Tests

Avant les possibles optimisations, les tests ont donné les résultats suivants :

- Tests run: 63, Failures: 0, Errors: 0, Skipped: 0
- Total time: 01:02 min

- Tests run: 9, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.051 s - in CSudoku.board.CSudokuBoardTest
- Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.452 s - in CSudoku.player.human.HumanPlayerTest
- Tests run: 16, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.179 s - in CSudoku.player.ai.EvaluatedSimulatedBoardTest
- Running CSudoku.player.ai.MinimaxMoveStrategyTest
Minimax: Evaluating 729 initial moves...
Recherche Minimax terminée:
Temps pris : 26740 ms
Meilleur coup trouvé: Move{row=0, col=0, value=9} | Score évalué : 9
- Minimax: Evaluating 729 initial moves...
- Recherche Minimax terminée:
- Temps pris : 26416 ms
- Meilleur coup trouvé: Move{row=0, col=0, value=9} | Score évalué : 9
- Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 53.178 s - in CSudoku.player.ai.MinimaxMoveStrategyTest
- Tests run: 9, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.216 s - in CSudoku.player.ai.AIPlayerTest
- Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.104 s - in CSudoku.player.automate.RandomMoveNoValidationStrategyTest
- Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.014 s - in CSudoku.player.automate.AutomatePlayerTest
- Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.065 s - in CSudoku.player.automate.FirstValidMoveStrategyTest
- Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.051 s - in CSudoku.player.automate.RandomMoveStrategyTest

*** *Les tests ont été lancés sur un ordinateur personnel assez faible*

- **Tests additionnels**

Des tests additionnels pour AlphaBeta et MCTS ont été ajoutés :

- Tests run: 71, Failures: 0, Errors: 0, Skipped: 0
- Running CSudoku.player.ai.AlphaBetaStrategyTest
AlphaBeta: Evaluating 729 initial moves at depth 3...
Recherche AlphaBeta terminée:
Noeuds visités :: 2526578
Coupes alpha: 720
Coupes beta 6979
Temps pris : 82743 ms
Meilleur coup trouvé: Move{row=0, col=0, value=9} | Score évalué : 9
AlphaBeta: Evaluating 729 initial moves at depth 3...
Recherche AlphaBeta terminée:
Noeuds visités :: 2526578
Coupes alpha: 720
Coupes beta 6979
Temps pris : 78492 ms
Meilleur coup trouvé: Move{row=0, col=0, value=9} | Score évalué : 9

- Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 161.246 s - in CSudoku.player.ai.AlphaBetaStrategyTest
- Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.014 s - in CSudoku.player.ai.MCTSStrategyTest

*** Les tests ont été lancés sur un ordinateur personnel assez faible

- **Performance et comparaisons**

1. AlphaBeta et Minimax contre un humain

On a lancé les tests sur un plateau de taille 9 en parallèle pour AlphaBeta et Minimax. Le premier coup était toujours celui du joueur 1 (humain), qui plaçait 9 sur les coordonnées (1,1). La recherche AlphaBeta montre des résultats plusieurs fois meilleurs grâce à la moindre quantité de nœuds visités. Les statistiques sont les suivantes :

AlphaBeta :

- Nœuds visités : 12662
- Nœuds visités (en totale): 12662
- Coupes alpha (en totale): 8513
- Coupes beta (en totale): 691
- Meilleur coup trouvé : Move{row=0, col=3, value=9} | Score évalué : 9
- Temps pris : 2082 ms

Minimax :

- Recherche Minimax terminée :
- Nœuds Visités : 470937
- Nœuds Visités (en total): 470937
- Meilleur coup trouvé : Move{row=0, col=3, value=9} | Score évalué : 9
- Temps pris : 36324 ms

On voit clairement que le nombre de nœuds est 37 fois moindre pour AlphaBeta et donc le temps est 18 fois moindre.

Pour le deuxième coup, où l'humain jouait 9 sur (8,7). Le temps 1 est 15 fois moindre pour AlphaBeta est 24 fois moindre. Pour le troisième coup, le temps devient 13 fois moindre. Ainsi, avec chaque coup, la différence dans le temps continue à diminuer, puisque la différence entre les nœuds visités continue à diminuer.

2. AlphaBeta vs Minimax

Pour toutes les simulations, c'est le premier joueur qui gagne avec le score 98 contre 90, mais AlphaBeta prend beaucoup moins de temps.

3. MCTS contre un humain

MCTS prend un peu moins de temps (par exemple 1500 ms contre 2000 ms pour AlphaBeta).

4. MCTS vs AlphaBeta, constante d'exploration

- Avec la constante d'exploration $\sqrt{2}$ actuelle de UCB1, il explore beaucoup les possibilités et peut commencer par un 5, puis choisir un 3. AlphaBeta choisit exclusivement les nombres les plus grands.
 - La partie s'arrêtant au milieu, le moins la constante d'exploration est (par exemple $\frac{\sqrt{2}}{2}$ ou 0.1, AlphaBeta gagne au général, mais la différence dans les scores devient de moins en moins grande. En augmentant la constante, la situation devient vice-versa.
- **Optimisations faites**
 1. AlphaBeta
 - Un tri de coups prend place dans sortValidMoves() pour trier les coups en fonction d'un score heuristique avant l'exploration
 - Réutilisation du plateau, (board.setValue() et board.unSet()) au lieu de créer une nouvelle instance de EvaluatedSimulatedBoard à chaque itération
 - Utilisation de Collections.sort()
 2. Minimax
 - Parallélisation des calculs
 - Table des transpositions pour stocker les coups déjà calculés
 - IDDFS (Iterative deepening depth-first search) avec augmentation de profondeur
 - Ordonnancement de coups
- **Tests après optimisations**
 1. Minimax
 - Total Time: 2001 ms
 - Total Nodes Visited: 546816
 - Le temps diminue de 18 fois
 - Le nombre de nœuds visités augmente
 2. AlphaBeta
 - Nœuds visités : 7453
 - Temps pris : 1079 ms
 - Diminution du temps de 2 fois
 - Le nombre de nœuds diminue 1.5 fois